

SUNIX SDC Expansion Board

SocketCAN

Sample Code Guideline

Driver Ver. 2.1.1.0

Doc Ver. 2.0

2024 August



SUNIX CO., Ltd.

Tel: +886-2-8913-1987

Fax: +886-2-8913-1986

<http://www.sunix.com>

info@sunix.com.tw

Index

1).	C/C++.....	3
1.1.	CAN2.0/CAN FD Sample	3
1.1.1.	Tutorial.....	3
1.1.2.	Sample Code	3
1.2.	J1939 Sample.....	7
1.2.1.	Tutorial.....	7
1.2.2.	Sample Code	7
2).	Python	11
2.1.	CAN2.0/CAN FD Sample	11
2.1.1.	Tutorial.....	11
2.1.2.	Sample Code	11
2.2.	J1939 Sample.....	20
2.2.1.	Tutorial.....	20
2.2.2.	Sample Code	20

1). C/C++

1.1. CAN2.0/CAN FD Sample

1.1.1. Tutorial

About SocketCAN C/C++ programming tutorial, please check kernel.org SocketCAN tutorial:

<https://docs.kernel.org/networking/can.html#>

1.1.2. Sample Code

➤ The test wire setting CAN0 ↔ CAN1.

1). Enter the SocketCAN C sample project directory.

```
./SDCLinuxExpansionBoardDriver_VX.X.X.X/SocketCAN/C/SocketCAN_test/
```

```
root@test-System-Product-Name: /home/test/SDCLinuxExpansionBo
root@test-System-Product-Name:/home/test/SDCLinuxExpansionBoardDriver_ /SocketCAN/C/SocketCAN_test# ls
function.h global.h main.c Makefile obj precomp.h socketcan_func_test.c utils.c
```

2). Compile the source code.

```
$make
```

```
root@test-System-Product-Name:/home/test/SDCLinuxExpansionBoardDriver_ /SocketCAN/C/SocketCAN_test# make
gcc -Wall -g -c main.c
gcc -Wall -g -c utils.c
gcc -Wall -g -c socketcan_func_test.c
mv -f *.o obj
gcc -g -o socketcan_test \
obj/main.o \
obj/utils.o \
obj/socketcan_func_test.o \
-L obj -pthread
rm -f obj/*.o
mv -f socketcan_test obj
```

3). Execute the sample program.

```
$/obj/socketcan_test
```

```
root@test-System-Product-Name:/home/test/SDCLinuxExpansionBoardDriver_ /SocketCAN/C/SocketCAN_test# ./obj/socketcan_test
```

```
-----
Date       : 2022/09/12
Version    : 0.0.1.0
-----

|                                     Socketcan R/W Example                                     |
=====
Usage before run program
Enable CAN device:
    sudo ip link set can0 type can bitrate 1000000
    sudo ip link set can0 up
or
Enable CAN FD device:
    sudo ip link set can0 type can bitrate 1000000 dbitrate 2000000 fd on
    sudo ip link set can0 up
=====
>-----<
A : Open
B : Close
C : Write CAN data
D : Write CAN FD data
E : Set receive filter
F : Clear receive filter

? : Show cmd
Q : quit
>-----<
d[#_#]b > CMD ?
```

- 4). Press 'A' key to open SocketCAN, input CAN channel name.

```
d[#_#]b > CMD ?

Please input CAN net device name(canxxx) ? can0
#INFO: can0 is running
#INFO: CAN_type: CAN FD
-----
#INFO: can0 rx thd create ok
-----
#INFO: can0 open ok, sd 3
-----
#INFO: Socketcan net device open Success
d[#_#]b > CMD ?
#INFO: can0 is waiting for CAN message
```

- 5). Received CAN FD frame

- can-utils: cansend.

```
$cansend can1 7ff##31a2b4c4d5e6b7c8f9a1b3c
```

```
root@test-System-Product-Name:~# cansend can1 7ff##31a2b4c4d5e6b7c8f9a1b3c
```

- socketcan_test received CAN FD frame.

```
can0 read data size=CANFD_MTU:72
-----can0, Receiving CAN FD message Start-----
#INFO: Got Standard frame
#INFO: Id=0x7FF Length=12
#INFO: Got FD_BRS
#INFO: Got FD_ESI
Data:
x1a x2b x4c x4d x5e x6b x7c x8f
x9a x1b x3c x00
-----can0, Receiving CAN FD message End-----
```

6). Press 'C' key to send CAN2.0 frame.

```
d[#_]b : 0->no, 1->yes
d[#_]b : user input ? 0
-----can0, Show write CAN message Start-----
id      : x00000001
len     : 8
-----data start-----
x01 x02 x03 x04 x05 x06 x07 x08
-----data end-----
-----can0, Show write CAN message End-----
#INFO: can0 write success
d[#_]b > CMD ?
```

➤ can-utils: candump.

```
$candump can1 -x -c
```

```
root@test-System-Product-Name:~# candump can1 -x -c
can1 RX - - 001 [8] 01 02 03 04 05 06 07 08
```

7). Press 'D' key to send CAN FD frame

```
-----can0, Show write CAN FD message Start-----
id      : x00000001
FD flags : 1
len     : 64
-----data start-----
x01 x02 x03 x04 x05 x06 x07 x08
x09 x0a x0b x0c x0d x0e x0f x10
x11 x12 x13 x14 x15 x16 x17 x18
x19 x1a x1b x1c x1d x1e x1f x20
x21 x22 x23 x24 x25 x26 x27 x28
x29 x2a x2b x2c x2d x2e x2f x30
x31 x32 x33 x34 x35 x36 x37 x38
x39 x3a x3b x3c x3d x3e x3f x40
-----data end-----
-----can0, Show write CAN FD message End-----
#INFO: can0 write success
```

- can-utils: candump

```
$candump can1 -x -c
```

```
root@test-System-Product-Name:~# candump can1 -x -c
can1 RX 8 - 001 [64] 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F 20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B
3C 3D 3E 3F 40
```

- 8). Press 'E' key to set filter.

```
d[#_#]b : range from 1 ~ 32
d[#_#]b : id filter amount#           ? 1
#1 id message filter
d[#_#]b : id (4 byte)                  ? x00000123
d[#_#]b : 0->standard, 1->extended
d[#_#]b : id filter frame type#       ? 0
#INFO: can0 Set receive filter success, file amount:1
-----#1 id message filter-----
id: x00000123
frame_type: Standard(0)
-----#1 id message filter-----
```

- Only accept ID:0x123, others are excluded.

```
-----can0, Receiving CAN FD message Start-----
#INFO: Got Standard frame
#INFO: Id=0x123 Length=12
#INFO: Got FD_BRS
#INFO: Got FD_ESI
Data:
x1a x2b x4c x4d x5e x6b x7c x8f
x9a x1b x3c x00
-----can0, Receiving CAN FD message End-----
```

- 9). Press 'F' key to clear filter.

```
#INFO: can0 Clear receive filter success
d[#_#]b > CMD ?
```

- 10). Press 'B' key to close SocketCAN.

```
d[#_#]b > CMD ?
#INFO: can0 is waiting for CAN message

-----
#INFO: can0 rx thd terminate ok
-----

#INFO: can0 close ok
-----

#INFO: Socketcan net device close Success
d[#_#]b > CMD ?
```

1.2. J1939 Sample

1.2.1. Tutorial

About SocketCAN J1939 C/C++ programming tutorial, please check kernel.org J1939 tutorial:

<https://docs.kernel.org/networking/j1939.html>

1.2.2. Sample Code

- The test wire setting CAN0 ↔ CAN1.

- 1). Enter the J1939 C sample project directory.

./SDCLinuxExpansionBoardDriver_VX.X.X.X/SocketCAN /C/SocketCAN_J1939_test/

```
root@test-System-Product-Name:/home/test/SDCLinuxExpansionBoardDriver_ /SocketCAN/C/SocketCAN_J1939_test# ls
function.h global.h J1939_test.c main.c Makefile obj precomp.h utils.c
```

- 2). Compile the source code.

\$make

```
root@test-System-Product-Name:/home/test/SDCLinuxExpansionBoardDriver_ /SocketCAN/C/SocketCAN_J1939_test# make
gcc -Wall -g -c main.c
gcc -Wall -g -c utils.c
gcc -Wall -g -c J1939_test.c
mv -f *.o obj
gcc -g -o J1939_test \
obj/main.o \
obj/utils.o \
obj/J1939_test.o \
-L obj -pthread
rm -f obj/*.o
mv -f J1939_test obj
```

- 3). For two ports J1939 transmission test, execute two sample programs.

`$/obj/J1939_test`

root@test-System-Product-Name:/home/test/SDCLinuxExpansionBoardDriver_ /SocketCAN/C/SocketCAN_J1939_test# `./obj/J1939_test`

```

-----
Date       : 2023/01/16
Version    : 0.0.1.0
-----
|                                     SocketCAN J1939 S/R Example                                     |
=====
>-----<
A : J1939 Socket Open
B : J1939 Socket Close
C : Send J1939 Message
D : Receive J1939 Message

? : Show cmd
Q : quit
>-----<
d[#_#]b > CMD ?

```

- 4). Enter 'A' to open CAN0 and CAN1 J1939 socket, input CAN channel name and J1939 source address.

- Program1 (Open CAN0 J1939 socket):

```

-----
Date       : 2023/01/16
Version    : 0.0.1.0
-----
|                                     SocketCAN J1939 S/R Example                                     |
=====
>-----<
A : J1939 Socket Open
B : J1939 Socket Close
C : Send J1939 Message
D : Receive J1939 Message

? : Show cmd
Q : quit
>-----<
d[#_#]b > CMD ?

Please input CAN net device name(canxxx) ? can0
Please input J1939 source address(note: 0xFF = J1939_NO_ADDR, 0xFE = J1939_IDLE_ADDR):d[#_#]b : SA (1 byte) ? x20
#INFO: can0 is running
#INFO: Set broadcast
-----
#INFO: can0 open ok, sd 3
-----
#INFO: J1939 net device open Success
d[#_#]b > CMD ?

```

- Program2 (Open CAN1 J1939 socket):


```

-----
Date       : 2023/01/16
Version    : 0.0.1.0
-----
|                                     |
|               SocketCAN J1939 S/R Example               |
|                                     |
-----
>-----<
A : J1939 Socket Open
B : J1939 Socket Close
C : Send J1939 Message
D : Receive J1939 Message

? : Show cmd
Q : quit
>-----<

d[#_#]b > CMD ?

Please input CAN net device name(canxxx) ? can1
Please input J1939 source address(note: 0xFF = J1939_NO_ADDR, 0xFE = J1939_IDLE_ADDR):d[#_#]b : SA (1 byte) ? x30
#INFO: can1 is running
#INFO: Set broadcast
-----
#INFO: can1 open ok, sd 3
-----

#INFO: J1939 net device open Success
d[#_#]b > CMD ?

```

5). Press 'D' key to create J1939 receive thread for waiting J1939 message.

➤ Program1

```

d[#_#]b > CMD ?

-----
#INFO: can0 J1939 receive thread create ok
-----

#INFO:J1939 receive thread starting...d[#_#]b > CMD ?
#INFO: can0 is waiting for J1939 message

```

➤ Program2

```

d[#_#]b > CMD ?

-----
#INFO: can1 J1939 receive thread create ok
-----

#INFO:J1939 receive thread starting...d[#_#]b > CMD ?
#INFO: can1 is waiting for J1939 message

```

6). Press 'C' key to send J1939 message with each other.

- Program1 CAN0 send J1939 message to CAN1, PGN: 61444 = F004_h, DA:20_h, Data: FF FF FF 68 13 FF FF FF.

```

Please input J1939 PGN:d[#_#]b : PGN (range:0~3FFFF) ? xF004
Please input J1939 destination address(note: 0xFF = J1939_NO_ADDR, 0xFE = J1939_IDLE_ADDR):d[#_#]b : DA (1 byte) ? x30
Input J1939 Data(8byte):
d[#_#]b : data (byte 7) ? xFF
d[#_#]b : data (byte 6) ? xFF
d[#_#]b : data (byte 5) ? xFF
d[#_#]b : data (byte 4) ? x68
d[#_#]b : data (byte 3) ? x13
d[#_#]b : data (byte 2) ? xFF
d[#_#]b : data (byte 1) ? xFF
d[#_#]b : data (byte 0) ? xFF
d[#_#]b > CMD ?

```

```

=====Received J1939 Message=====
Address: 0x20
PGN: 0xF004
Data:0xFF 0xFF 0xFF 0x68 0x13 0xFF 0xFF 0xFF
=====

```

- Program2 CAN1 send J1939 message to CAN0, PGN: 65262 = $FEFE_h$,
DA: 20_h , Data: FF FF FF 00 FB FF FF FF

```

Please input J1939 PGN:d[#_#]b : PGN (range:0~3FFFF) ? xFEFE
Please input J1939 destination address(note: 0xFF = J1939_NO_ADDR, 0xFE = J1939_IDLE_ADDR):d[#_#]b : DA (1 byte) ? x20
Input J1939 Data(8byte):
d[#_#]b : data (byte 7) ? xFF
d[#_#]b : data (byte 6) ? xFF
d[#_#]b : data (byte 5) ? xFF
d[#_#]b : data (byte 4) ? x00
d[#_#]b : data (byte 3) ? xFB
d[#_#]b : data (byte 2) ? xFF
d[#_#]b : data (byte 1) ? xFF
d[#_#]b : data (byte 0) ? xFF
d[#_#]b > CMD ?

```

```

=====Received J1939 Message=====
Address: 0x30
PGN: 0xFEFE
Data:0xFF 0xFF 0xFF 0x00 0xFB 0xFF 0xFF 0xFF
=====

```

7). Close the two channel J1939 socket and terminate the receive thread.

- Program1, close can0 socket and terminate the receive thread:

```

-----
#INFO: can0 J1939 receive thread terminate ok
-----
#INFO: can0 close ok
-----
#INFO: J1939 net device close Success
d[#_#]b > CMD ?

```

- Program2, close can1 socket and terminate the receive thread:

```
-----  
#INFO: can1 J1939 receive thread terminate ok  
-----  
  
-----  
#INFO: can1 close ok  
-----  
  
#INFO: J1939 net device close Success  
d[#_#]b > CMD ?  
█
```

2). Python

2.1. CAN2.0/CAN FD Sample

2.1.1. Tutorial

The Python SocketCAN sample code is implemented by **python-can** module.
About the programming tutorial, please check [readthedocs.org](https://python-can.readthedocs.org/):

<https://python-can.readthedocs.io/en/stable/>

2.1.2. Sample Code

- 1). Install python3.

```
$apt-get install python3
```

- 2). Install pip.

```
$apt-get install python3-pip
```

- 3). Install python-can module by pip.

```
$pip3 install python-can
```

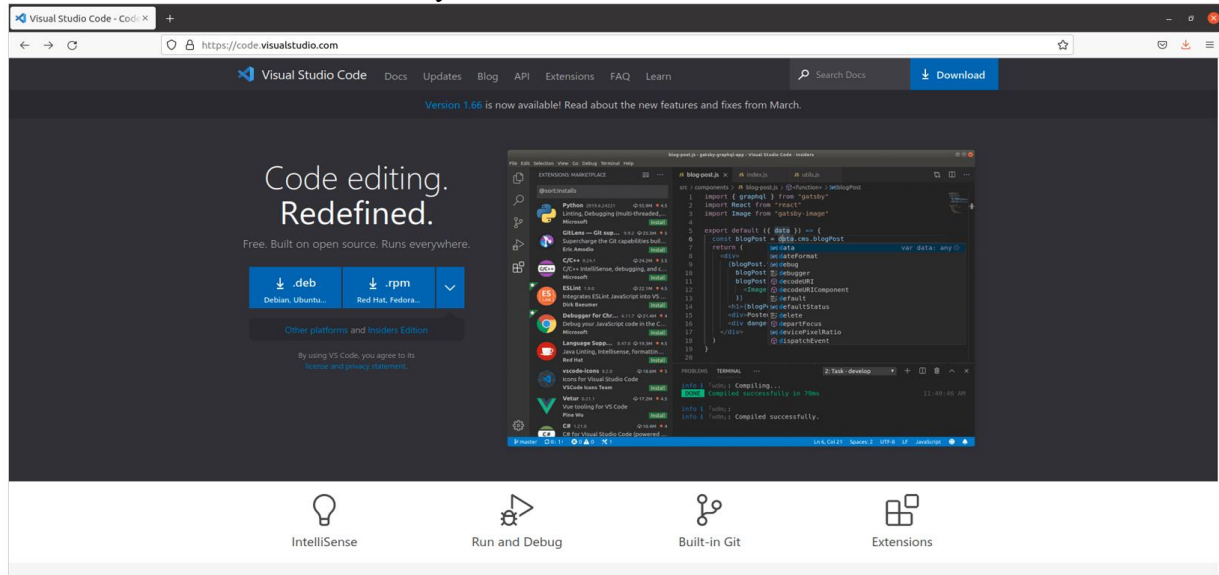
- 4). Install keyboard module by pip

```
$pip3 install keyboard
```

5). Install click module by pip

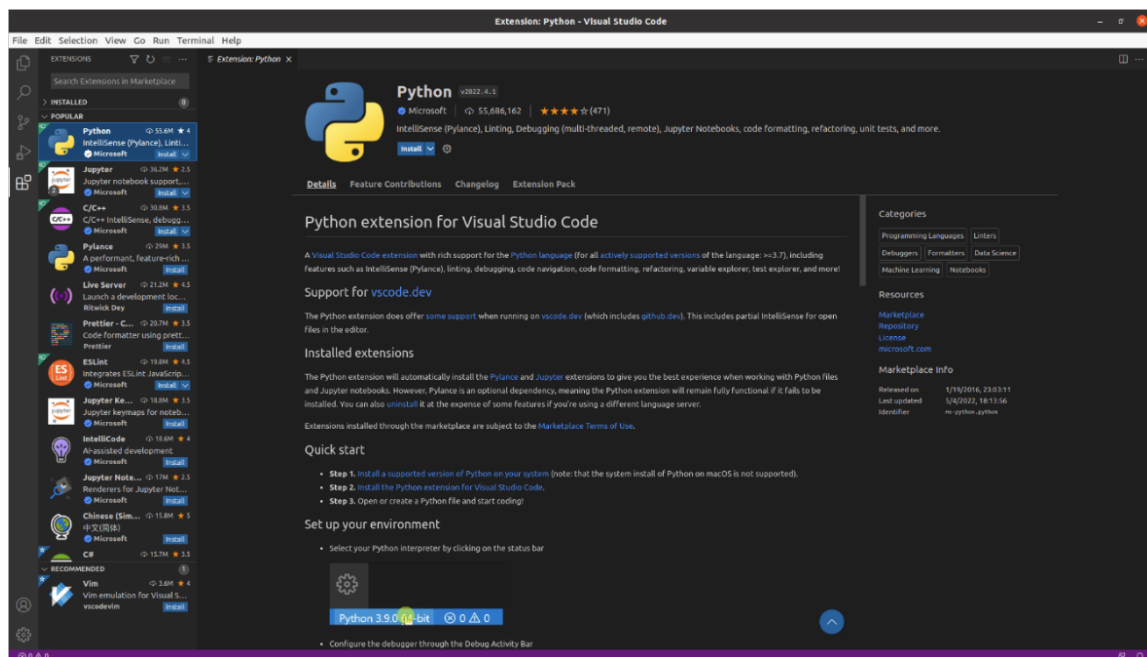
\$pip3 install click

6). Install IDE, This case use Visual Studio Code as example, developer could download the deb file by official website and install it.



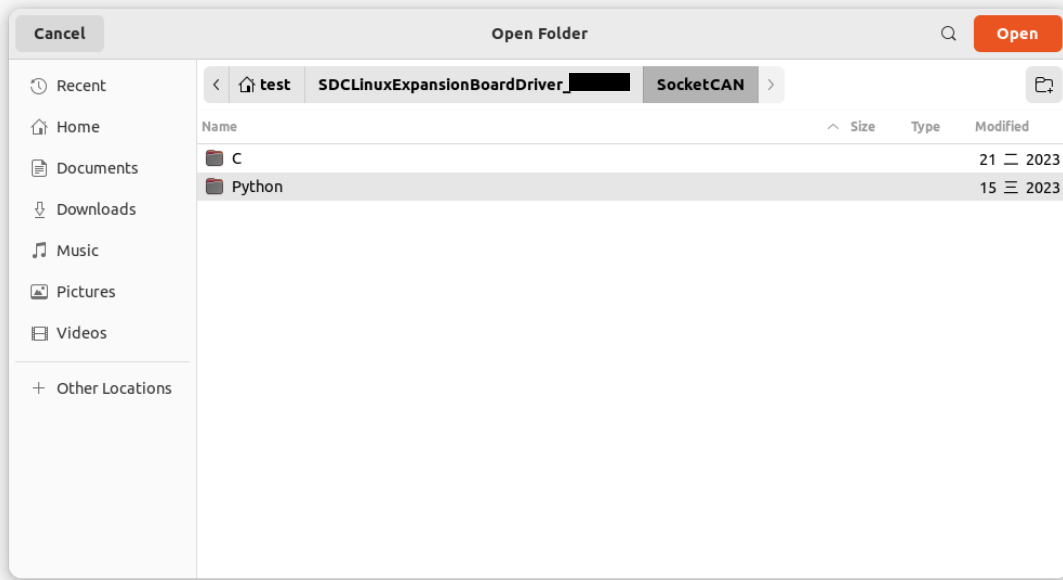
Anyway, developer can choose IDE according to your preferences, e.g. PyCharm, Spyder, Atom, Eclipse, or even vim.

7). Install the Python development kit for VScode.

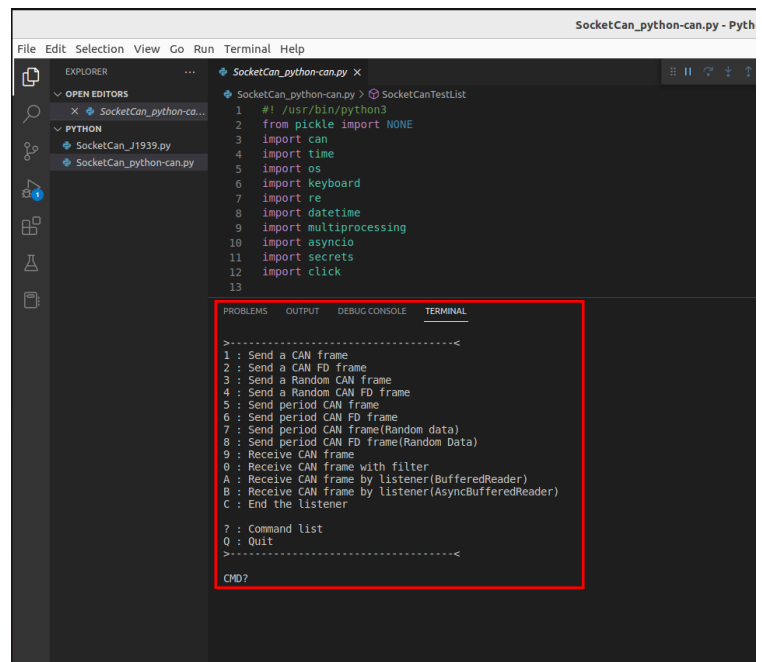
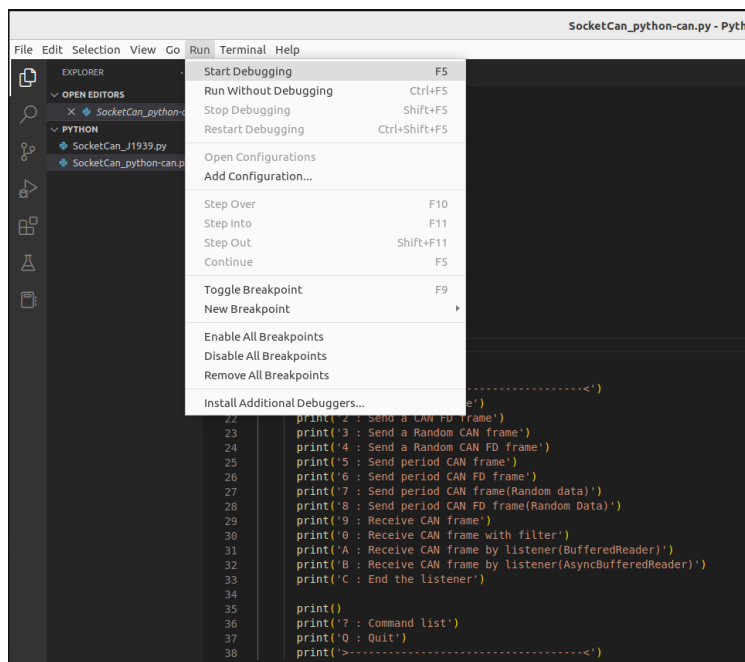


- 8). Open the folder which content the SocketCan_python-can.py sample
The directory:

./SDCLinuxExpansionBoardDriver_VX.X.X.X /SocketCAN/Python/SocketCan_python-can.py



- 9). Start to debug SocketCan_python-can.py.



- 10). Stop VSCode debug, run SocketCan_python-can.py on terminal.

```
./SocketCan_python-can.py
```

```
root@test-System-Product-Name:/home/test/SDCLinuxExpansionBoardDriver_ /SocketCAN/Python# ls
SocketCan_J1939.py SocketCan_python-can.py
root@test-System-Product-Name:/home/test/SDCLinuxExpansionBoardDriver_ /SocketCAN/Python# ./SocketCan_python-can.py
```

- The test wire setting CAN0 ↔ CAN1.

```
>-----<
1 : Send a CAN frame
2 : Send a CAN FD frame
3 : Send a Random CAN frame
4 : Send a Random CAN FD frame
5 : Send period CAN frame
6 : Send period CAN FD frame
7 : Send period CAN frame(Random data)
8 : Send period CAN FD frame(Random Data)
9 : Receive CAN frame
0 : Receive CAN frame with filter
A : Receive CAN frame by listener(BufferedReader)
B : Receive CAN frame by listener(AsyncBufferedReader)
C : End the listener

? : Command list
Q : Quit
>-----<
```

- 11). Press '1' key to send CAN2.0 frame.

```
CMD?
1
Channel?(eg: "can0" -> input 0)
0
ID?(Base:11bits; Extended:29bits)
7ff
Data? <= 8bytes(e.g. 12 34 56 78 90 AB CD EF)
12 34 56 78 90 ab cd ef
***** Message Sent *****
Timestamp:2022-09-29 11:25:55.478739
Channel:None
ID:0x7ff
DLC:8
Data:12 34 56 78 90 AB CD EF
FD:False
*****
```

```
root@test-System-Product-Name:~# candump can1 -x -c
can1 RX - - 7FF [8] 12 34 56 78 90 AB CD EF
```

12). Press '2' key to send CAN FD frame.

```
CMD?
2
Channel?(eg: "can0" -> input 0)
0
ID?(Base:11bits; Extended:29bits)
1fff
Data? <= 64bytes(e.g. 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF)
12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
***** Message Sent *****
Timestamp:2022-09-29 11:35:53.298006
Channel:None
ID:0x1fff
DLC:16
Data:12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
FD:True
BRS:True
ESI:False
*****
```

```
root@test-System-Product-Name:~# candump can1 -x -c
can1 RX B - 00001FFF [16] 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
```

13). Press '5' key to send period CAN frames. Send once per second, for a total of ten seconds.

```
CMD?
3
Channel?(eg: "can0" -> input 0)
0
ID?(Base:11bits; Extended:29bits)
7ff
Data? <= 8bytes(e.g. 12 34 56 78 90 AB CD EF)
12 34 56 78 90 AB CD EF
Period?(period in seconds between each message)
1
Duration?(in seconds to continue sending messages)
10
***** Period Message Sent *****
Timestamp:2022-09-29 11:44:10.946930
Period:1sec
Duration:10sec
Channel:None
ID:0x7ff
DLC:8
Data:12 34 56 78 90 AB CD EF
FD:False
*****
```



```
root@test-System-Product-Name:~# candump can1 -x -c
can1 RX - - 7FF [8] 12 34 56 78 90 AB CD EF
can1 RX - - 7FF [8] 12 34 56 78 90 AB CD EF
can1 RX - - 7FF [8] 12 34 56 78 90 AB CD EF
can1 RX - - 7FF [8] 12 34 56 78 90 AB CD EF
can1 RX - - 7FF [8] 12 34 56 78 90 AB CD EF
can1 RX - - 7FF [8] 12 34 56 78 90 AB CD EF
can1 RX - - 7FF [8] 12 34 56 78 90 AB CD EF
can1 RX - - 7FF [8] 12 34 56 78 90 AB CD EF
can1 RX - - 7FF [8] 12 34 56 78 90 AB CD EF
can1 RX - - 7FF [8] 12 34 56 78 90 AB CD EF
```

- 14). Press '6' key to send period CAN FD frames. Send once per second, for a total of ten seconds.

```
CMD?
4
Channel?(eg: "can0" -> input 0)
0
ID?(Base:11bits; Extended:29bits)
1ffff
Data? <= 64bytes(e.g. 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF)
12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
Period?(period in seconds between each message)
1
Duration?(in seconds to continue sending messages)
10
***** Period Message Sent *****
Timestamp:2022-09-29 11:47:14.438127
Period:1sec
Duration:10sec
Channel:None
ID:0x1ffff
DLC:16
Data:12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
FD:True
BRS:True
ESI:False
*****
```

```
root@test-System-Product-Name:~# candump can1 -x -c
can1 RX B - 0001FFFF [16] 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
can1 RX B - 0001FFFF [16] 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
can1 RX B - 0001FFFF [16] 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
can1 RX B - 0001FFFF [16] 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
can1 RX B - 0001FFFF [16] 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
can1 RX B - 0001FFFF [16] 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
can1 RX B - 0001FFFF [16] 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
can1 RX B - 0001FFFF [16] 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
can1 RX B - 0001FFFF [16] 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
can1 RX B - 0001FFFF [16] 12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
```

- Option 3,4,7,8 just using random to generate data, the other functions are the same, so ignore it here.

- 15). Press '9' key to start the thread to receive frame, click 'esc' key to end it.

```
root@test-System-Product-Name:~# cansend can1 7ff##31234567890abcdef1234567890abcdef
```



```

CMD?
Channel?(eg: "can0" -> input 0)
0
CAN2.0/FD?(0:CAN 2.0; 1:CAN FD)
1
Start receiving data, press "esc" key to end receive mode.
***** Received Message *****
Timestamp:2023-03-15 15:11:51.458838
Channel:can0
ID:0x7ff
DLC:16
Data:12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
FD:True
BRS:True
ESI:True
*****
*****press "esc" key to end the receive mode*****
*****

```

16). Press '0' key to received can frame with filter

➤ Filters:

```

Filters = [{"can_id":0x123, "can_mask": 0x7FE, "extended": False},
           {"can_id":0xA0000, "can_mask": 0x1FFFFFFF, "extended": True}]

```

```

CMD?
6
Tip: Socket can filter formula: received_can_id & mask = code_id & mask
Channel?(eg: "can0" -> input 0)
0
CAN2.0/FD?(0:CAN 2.0; 1:CAN FD)
1
Set a base code ID for base filter?
123
Set a base mask for base filter?
7FE
Set a extended code ID for extended filter?
A0000
Set a extended mask for extended filter?
1FFFFFFF
Start receiving data, press "esc" key to end receive mode.

```

➤ Test Result: the base id only accept 0x122 & 0x123; the extended id only accept 0xA0000.

```

root@test-System-Product-Name:~# cansend can1 123##31234567890abcdef1234567890abcdef
root@test-System-Product-Name:~# cansend can1 122##31234567890abcdef1234567890abcdef

```

```
***** Received Message *****
Timestamp:2022-09-29 13:35:20.028921
Channel:can0
ID:0x123
DLC:16
Data:12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
FD:True
BRS:True
ESI:True
***** Received Message *****
Timestamp:2022-09-29 13:35:34.142998
Channel:can0
ID:0x122
DLC:16
Data:12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
FD:True
BRS:True
ESI:True
*****
```

```
***** Received Message *****
Timestamp:2022-09-29 13:38:51.363847
Channel:can0
ID:0xa0000
DLC:16
Data:12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
FD:True
BRS:True
ESI:True
*****
```

- About the SocketCAN filter tutorial, please check Kernel.org SocketCAN document → RAW socket option CAN_RAW_FILTER:
<https://docs.kernel.org/networking/can.html#>

17). Press 'A' key to received frames by **Listener's** subclass method:

BufferedReader

```
CMD?
7
Channel?(eg: "can0" -> input 0)
0
CAN2.0/FD?(0:CAN 2.0; 1:CAN FD)
1
can0 start listening...
```

```
root@test-System-Product-Name:~# cansend can1 1FFFFFFF##31234567890abcdef1234567890abcdef
```

```
***** Message Notification *****
Timestamp:2022-09-29 13:42:55.063862
Channel:can0
ID:0x1fffffff
DLC:16
Data:12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
FD:True
BRS:True
ESI:True
*****
```

18). Press 'C' key to end listening

```
CMD?
9
End listening...
```

19). Press 'B' key to received frames by Listener's subclass method
AsyncBufferedReader.

```
CMD?
8
Channel?(eg: "can0" -> input 0)
0
CAN2.0/FD?(0:CAN 2.0; 1:CAN FD)
1
/usr/lib/python3.10/multiprocessing/process.py:125: RuntimeWarning: coroutine 'AsyncCanListen' was never awaited
  del self._target, self._args, self._kwargs
RuntimeWarning: Enable tracemalloc to get the object allocation traceback
can0 start asynchronous listening ...
```

```
root@test-System-Product-Name:~# cansend can1 1FFFFFFF##31234567890abcdef1234567890abcdef
```

```
CMD?
***** Async Message Notification *****
Timestamp:2022-09-29 14:04:15.627486
Channel:can0
ID:0x1fffffff
DLC:16
Data:12 34 56 78 90 AB CD EF 12 34 56 78 90 AB CD EF
FD:True
BRS:True
ESI:True
*****
```

20). Press '9' key to end listening

```
CMD?
9
End listening...
```

- The python-can provide socket listener that make the process of receiving data more stable and efficient.
- About the **listener** and **asynchronous programming**, please check the python.org and readthedocs.org :
- asyncio

<https://docs.python.org/3/library/asyncio.html>

- python-can Listener class and Notifier class

<https://python-can.readthedocs.io/en/stable/notifier.html>

2.2. J1939 Sample

2.2.1. Tutorial

The Python J1939 sample code is implemented by **j1939** module.

About the programming tutorial, please check readthedocs.org and GitHub:

<https://j1939.readthedocs.io/en/latest/readme.html#overview>

<https://github.com/juergenH87/python-can-j1939>

2.2.2. Sample Code

- 1). Install python3.

```
$apt-get install python3
```

- 2). Install pip.

```
$apt-get install python3-pip
```

- 3). Install can-j1939 module by pip.

```
$pip3 install can-j1939
```

➤ Note: The module name should not be 'j1939', this is an old version of the module, and there will be errors.

- 4). Install keyboard module by pip

```
$pip3 install keyboard
```

- 5). Install click module by pip

```
$pip3 install click
```

- 6). Run SocketJ1939.py on terminal, for two ports J1939 transmission test, execute **two sample program**.

./ SDCLinuxExpansionBoardDriver_VX.X.X.X /SocketCAN/Python

\$/SocketCan_J1939.py

root@test-System-Product-Name:/home/test/SDCLinuxExpansionBoardDriver_ /SocketCAN/Python# ./SocketCan_J1939.py

- The test wire setting CAN0 $\leftrightarrow$ CAN1.

```
>-----<
1 : J1939 Socket Open
2 : J1939 Socket Close
3 : Send J1939 Message
4 : Receive J1939 Message

? : Command list
Q : Quit
>-----<

CMD?
█
```

- 7). Press '1' key to open CAN0 and CAN1 J1939 socket, input CAN channel number.

- Program1 (Open CAN0 J1939 socket):

```
>-----<
1 : J1939 Socket Open
2 : J1939 Socket Close
3 : Send J1939 Message
4 : Receive J1939 Message

? : Command list
Q : Quit
>-----<

CMD?
1
Channel?(eg: "can0" -> input 0)
0
can0 J1939 socket open sucess

CMD?
█
```

- Program2 (Open CAN1 J1939 socket):

```

>-----<
1 : J1939 Socket Open
2 : J1939 Socket Close
3 : Send J1939 Message
4 : Receive J1939 Message

? : Command list
Q : Quit
>-----<

CMD?
1
Channel?(eg: "can0" -> input 0)
1
can1 J1939 socket open sucess

CMD?

```

8). CAN1 press '4' key to create callback function for waiting J1939 message.

➤ Program 2

```

>-----<
1 : J1939 Socket Open
2 : J1939 Socket Close
3 : Send J1939 Message
4 : Receive J1939 Message

? : Command list
Q : Quit
>-----<

CMD?
1
Channel?(eg: "can0" -> input 0)
1
can1 J1939 socket open sucess

CMD?
4
can1 waitng for J1939 message ...

CMD?

```

9). CAN0 press '1' key to send J1939 message to CAN1.

➤ Program 1, PGN: 61444 = F004_h, DA: 30_h, Data: FF FF FF 68 13 FF FF FF

```
>-----<
1 : J1939 Socket Open
2 : J1939 Socket Close
3 : Send J1939 Message
4 : Receive J1939 Message

? : Command list
Q : Quit
>-----<

CMD?
1
Channel?(eg: "can0" -> input 0)
0
can0 J1939 socket open sucess

CMD?
3
PGN? (Hex format, range: 0x0 ~ 0x3ffff)
F004
Address(SA, Hex format, 1 byte)?
30
Data? <= 8bytes(e.g. 12 34 56 78 90 AB CD EF)
FF FF FF 68 13 FF FF FF

CMD?

```

- Program 2, CAN1 received the J1939 message.

```
***** Received J1939 Message *****
Timestamp:2023-03-14 17:37:08.921326
Priority:6
Source Address:30
PGN:61444, hex format:F004
Data:FF FF FF 68 13 FF FF FF
*****
```